Dynamic Data Structure : linked list

We have discussed dynamic memory management using malloc, free and other functions in the last notes. We will continue from where we left and we will discuss some examples of use of dynamic memory management.

In these examples, models or diagrams are very necessary to assist us in understanding the concepts. Please draw the diagrams for yourself while reading these notes and understand how these programs work. It is extremely important to understand where the pointers point to and how they get changed.

Let us examine the code in 1_ex.c.

struct X is a type. struct Y has a variable of struct X within it. It is a classical

example of structure within a structure.

There is an issue with struct Z.

When the compiler compiles the struct defnition, it decides the **binary layout of the structure**. It decides the relative position or offset of each field and also decides the total size of the structure – with or without padding based on the **alignment** criteria.

```
struct Z
{
        int c;
        struct Z z; // infnitely recursively included
};
```

We cannot fnd the size of struct Z as we cannot fnd the size of its field z which itself is a structure of the same kind. This results in a compile time error.

Struct A is perfectly alright. The size of pointer to any type on a given implementation is fixed. So size of the structure can be computed by the compiler at compile time.

A structure which has a pointer to itself as a field is called a self referential

structure.

```
struct A
{
	int c;
	struct A *p;
};
```

// 1_ex.c

```
#include <stdio.h>
#include <stdlib.h>

struct X
{
	int a;
};

struct Y
{
	int b;
	struct X x;
};

#if 0
struct Z
{
	int c;
	struct Z z; // infnitely recursively included
};
#endif

// self referential structure
struct A
{
	int c;
	struct A *p;
};
```

```
int main()
{
	printf("size : %lu\n", sizeof(struct X));
	printf("size : %lu\n", sizeof(struct Y));
	//printf("size : %lu\n", sizeof(struct Z));
	printf("size : %lu\n", sizeof(struct A));
	struct A one;
	one.c = 10;
	one.p = &one;
	printf("%d %d %d\n", one.c, one.p->c, one.p->p->c);
}
```

In this example, pointer p of the variable one points to variable one.

So, one→p is same as one→p→p !!.

Linked List: a simple example:

A List is a data structure like an array. It has the following characteristics.

Has zero or more components

Each component has an ordinal position. We can refer to an element by its position. You may recall that set of Python does not support this concept.

We can access element one after another. There is no way to access the element directly as in the case of arrays. Arrays have random or direct access whereas lists have sequential access.

Insertion and deletion in a list at a given position requires a very few operations and requires no shifting.

A Linked list is an implementation of list using self referential structures.

Let us discuss the following piece of code.

```
struct node
{
        int key_;
        struct node *next_;
};
typedef struct node node_t;
```

So, struct node has a field called link whose type is a pointer to the same

structure. struct node is a classical example of self-referential structure.

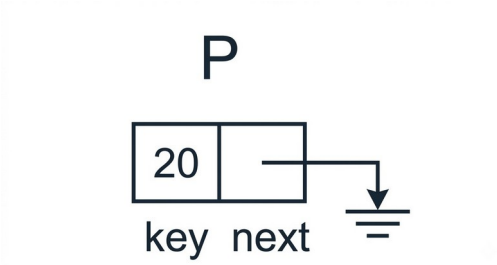
p is a pointer to structure.

node_t *p;

p = (node_t*)malloc(sizeof(node_t));

p->key_ = 20;

p points to a structure dynamically created. p→key_ has been assigned a value.



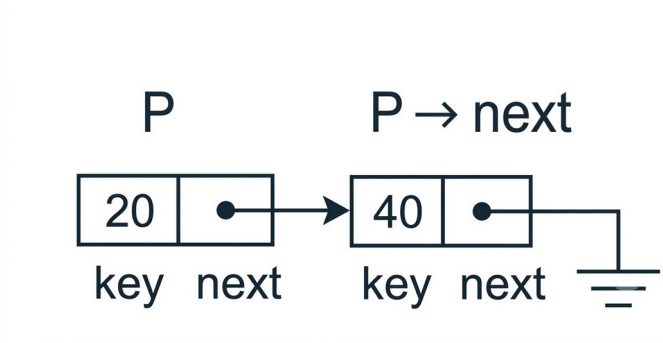
One more structure is created and p→next_ points to it. Key field in this new

structure is assigned 40.

Check the diagram.

p->next_ = (node_t*)malloc(sizeof(node_t));

p->next_->key_ = 40;



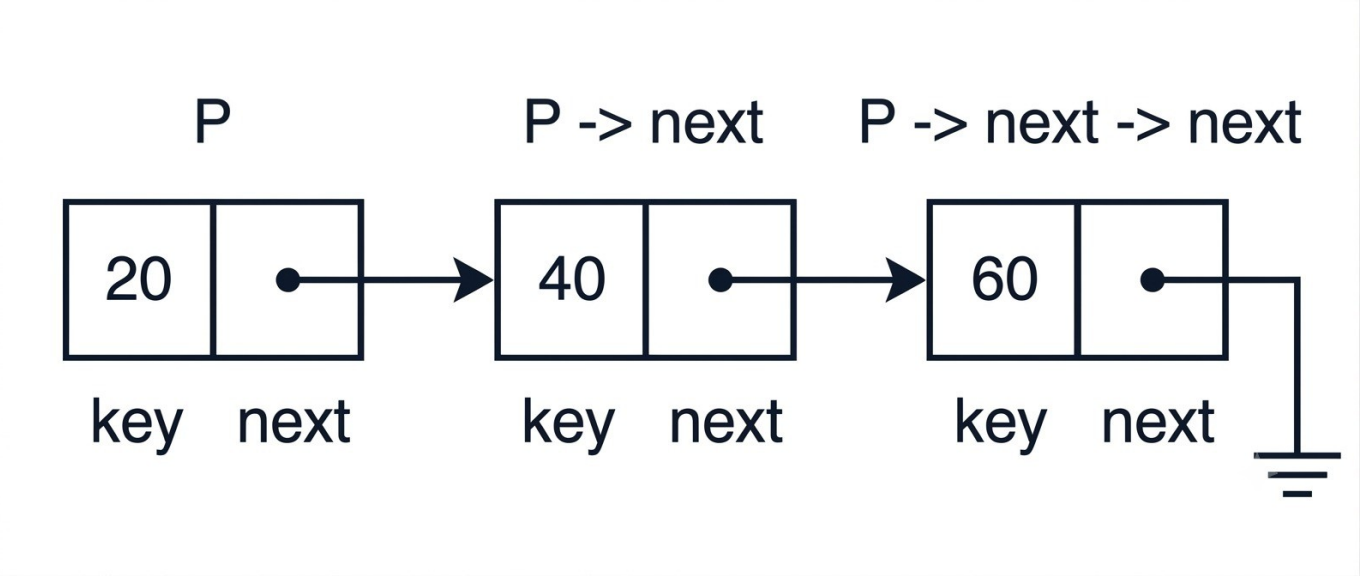
One more structure is created. Link field of the earlier structure points to this

Link field of the new structure is grounded (assigned NULL). So, the list ends.

```
p->next_->next_ = (node_t*)malloc(sizeof(node_t));

p->next_->next_->key_ = 60;

p->next_->next_->next_ = NULL;
```



```
// 2_ex.c

#include <stdio.h>

#include <stdlib.h>

struct node
{
	int key_;

	struct node *next_;
};
typedef struct node node_t;

void disp(node_t* q)
{
	while(q != NULL)
	{
		printf("%d ", q->key_);

		q = q->next_;

	}
}



void freelist(node_t* q)
{
	node_t* r;

	while(q != NULL)

	{

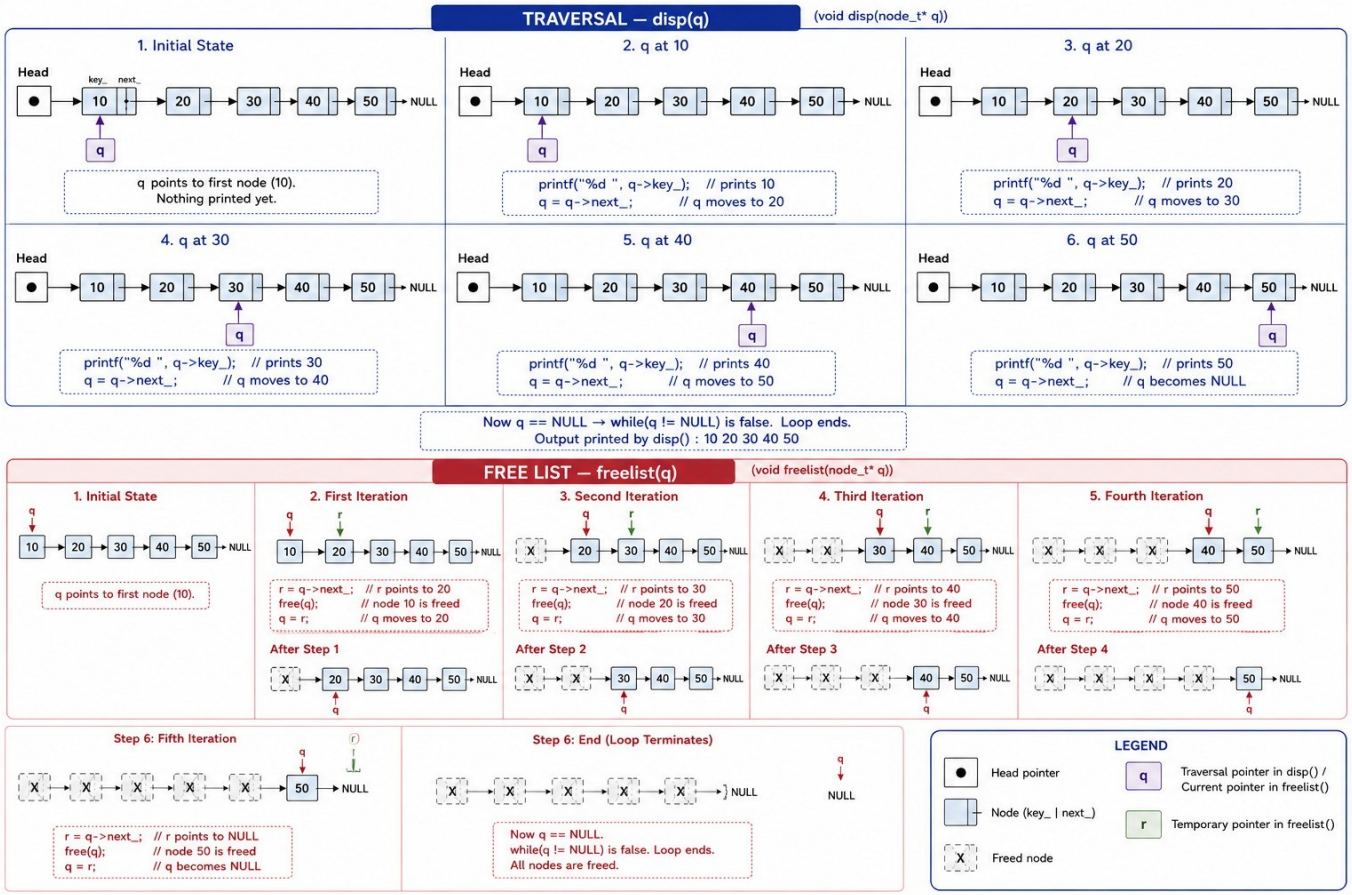
		r = q->next_;

		free(q);

		q = r;

	}

}
```


TRAVERSAL — disp(q)
(void disp(node_t* q))
1. Initial State
Head
key_ next_
10 20 30 40 50 NULL
q
q points to first node (10).
Nothing printed yet.
2. q at 10
printf("%d ", q->key_); // prints 10
q = q->next_; // q moves to 20
3. q at 20
printf("%d ", q->key_); // prints 20
q = q->next_; // q moves to 30
4. q at 30
printf("%d ", q->key_); // prints 30
q = q->next_; // q moves to 40
5. q at 40
printf("%d ", q->key_); // prints 40
q = q->next_; // q moves to 50
6. q at 50
printf("%d ", q->key_); // prints 50
q = q->next_; // q becomes NULL
Now q == NULL → while(q != NULL) is false. Loop ends.
Output printed by disp() : 10 20 30 40 50
FREE LIST — freelist(q)
(void freelist(node_t* q))
1. Initial State
q points to first node (10).
2. First Iteration
r = q->next_; // r points to 20
free(q); // node 10 is freed
q = r; // q moves to 20
After Step 1
3. Second Iteration
r = q->next_; // r points to 30
free(q); // node 20 is freed
q = r; // q moves to 30
After Step 2
4. Third Iteration
r = q->next_; // r points to 40
free(q); // node 30 is freed
q = r; // q moves to 40
After Step 3
5. Fourth Iteration
r = q->next_; // r points to 50
free(q); // node 40 is freed
q = r; // q moves to 50
After Step 4
Step 6: Fifth Iteration
r = q->next_; // r points to NULL
free(q); // node 50 is freed
q = r; // q becomes NULL
Step 6: End (Loop Terminates)
Now q == NULL.
while(q != NULL) is false. Loop ends.
All nodes are freed.
LEGEND
Head pointer
Node (key_ | next_)
Freed node
Traversal pointer in disp() / Current pointer in freelist()
Temporary pointer in freelist()

```
int main()
{
        node_t *p;
        p = (node_t*)malloc(sizeof(node_t));
        p->key_ = 20;
        p->next_ = (node_t*)malloc(sizeof(node_t));
        p->next_->key_ = 40;

        p->next_->next_ = (node_t*)malloc(sizeof(node_t));
        p->next_->next_->key_ = 60;
        p->next_->next_->next_ = NULL;
        disp(p);
        freelist(p);
}
```

p points to the beginning of the list - points to the frst node of the list. p is referred as the head of the list. Given the head of the list, we can traverse or walk through the list. Let us examine the disp function. The parameter q gets a copy of p. q points to the frst node in the list.

printf("%d ", q→key_);

q→key_ displays the key field in the frst node.

We should move to the next node. We cannot do q++ as the nodes are not necessarily allocated contiguous memory unlike arrays. The link field in the node to which q points to, points to the next

node. So, we copy the link field of q to q. The statement below is the most important statement in data

structures.

```
q = q→next_;
```

we repeat this until q becomes NULL signifying that we have come to the end of the list.

The function freelist releases the memory occupied by the linked list.

Observe that we should not say

free(p);

This would release the frst node and rest of the list becomes garbage.

```
    r = q->next_;
    free(q);
    q = r;
```

Observe that before removing what q points to, we should copy the link field so that we can change q to point to the next node after deleting what q pointed to before. The order of statements is very important. This technique of storing a pointer value temporarily before restoring it into some variable is very commonly used in data structures.

Ordered list:

// client.c

An ordered list has nodes arranged in the order of key based on some predicate. Let us try to create a list of nodes in the non-decreasing order of keys.

The list structure is opaque for the user. The user should not directly access anything within this structure as this could change.

Our implementation of list structure has the head of the list within it. Our implementation uses self referential structure node_t.

The client code initializes the list. The empty list normally grounds the head – makes it NULL.

The client code inserts elements into the list by calling the function insert_list.

The insert_list function creates a node, fills the key field, grounds the link field and then inserts the node in the right position in the list.

There are four cases to consider.

a) empty list

b) insert in the middle

c) insert at the end

d) insert in the beginning

```
if(ptr_list->head == NULL)
{
        ptr_list->head = temp;
        temp->next_ = NULL;
}
```

The code above takes care of the empty list case. The head being NULL indicates that the list is empty. Change the head to point to the new node created and ground the pointer of the new node.

To insert in a non-empty list, we have to find the position to insert. We make a pointer pres point to the first node of the list. Compare the key in node pointed to by pres to the key in new node pointed to by temp. if the former is less than the latter, we move to the next node by changing pres. pres = pres→next_. We realize that we should have inserted only after we have crossed the point. **It is like traveling in one way traffic**. We cannot move back.

We can overcome this problem by having a prev pointer tail before the pres as

shown in the code. The initial value of prev is NULL.

```
node_t* prev = NULL;

node_t* pres = ptr_list->head;

while(pres != NULL && pres->key_ < temp->key_ )

{

        prev = pres;

        pres = pres->next_;

}
```

We exit the loop in two ways. Either we have found a position to insert or the we have reached the end of the list.

In case of insertion in the beginning of the list, head has to be changed. We make out that the insertion has to be in the beginning, if the prev is still NULL after the loop is exited.

```
if(prev == NULL) // insert in the beginning

{

        temp->next_ = pres;

        ptr_list->head = temp;

}

else // middle or end

{

        temp->next_ = pres;

        prev->next_ = temp;

}
```

We also note that the code for insertion at the end or in the middle is same.

We insert between the nodes pointed to by prev and pres. Next_ of prev points to what temp points to and link of temp points to what pres points to.

The rest of the functions are self explanatory.

```
#include <stdio.h>

#include "mylist.h"

int main()

{

	mylist_t l;

	init_list(&l);

	int a[] = { 20, 10, 50, 30, 40};

	int n = 5;

	for(int i = 0; i < n; ++i)

	{

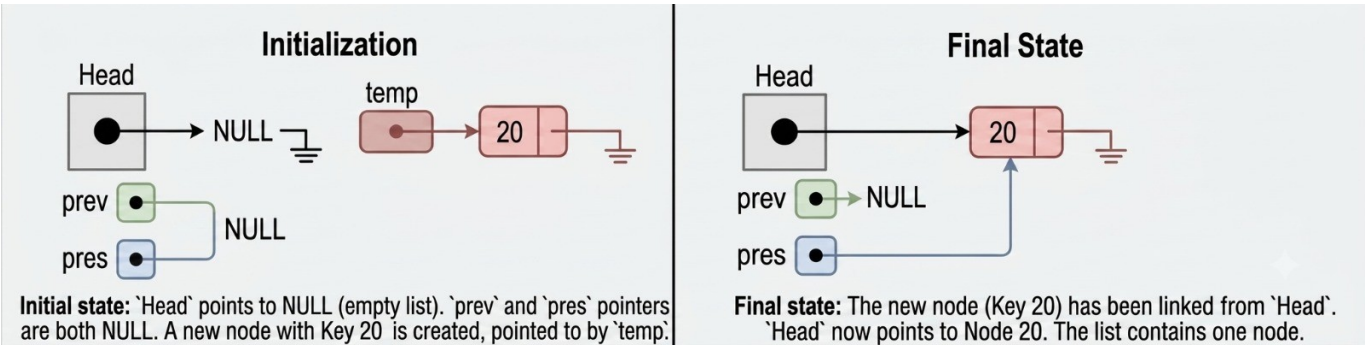
		insert_list(&l, a[i]);

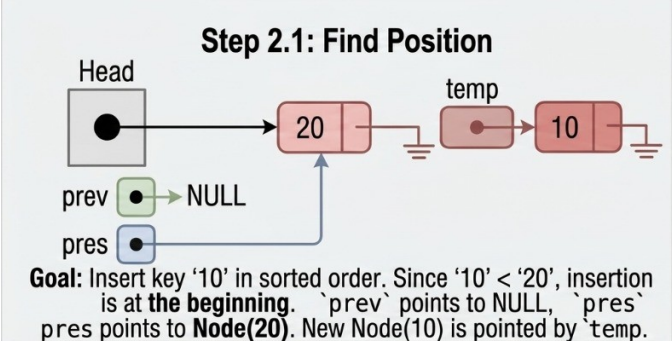
	}

		disp_list(&l);

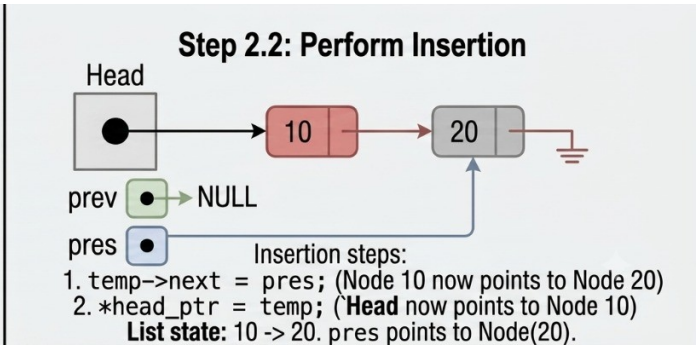
		free_list(&l);

}
```



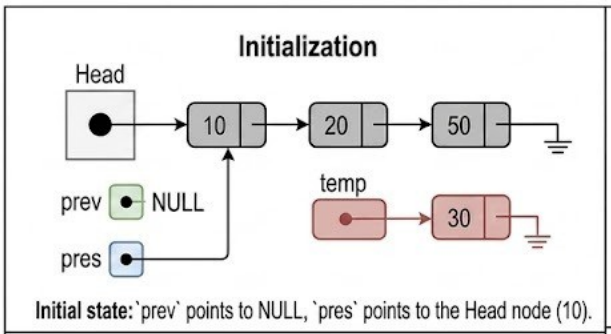


Goal: Insert key '10' in sorted order. Since '10' < '20', insertion is at **the beginning**. `prev` points to NULL, `pres` `pres` points to **Node(20)**. New Node(10) is pointed by `temp`.

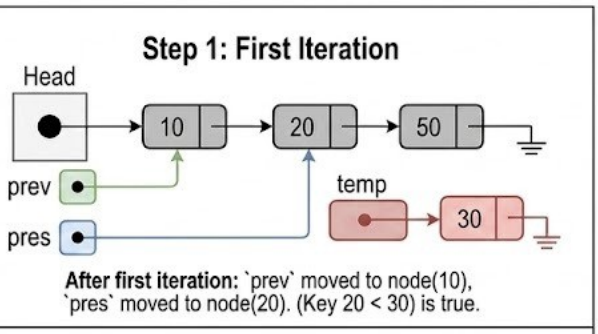


Insertion steps:
1. `temp->next = pres;` (Node 10 now points to Node 20)
2. `*head_ptr = temp;` (`**Head** now points to Node 10)

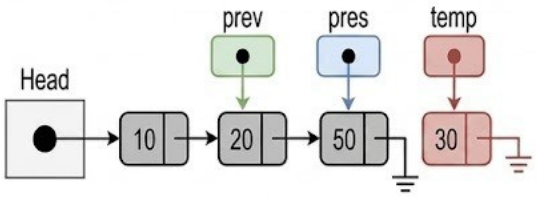
List state: 10 -> 20. `pres` points to Node(20).



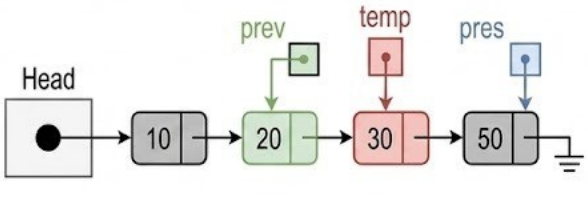
Initial state: `prev` points to NULL, `pres` points to the Head node (10).



After first iteration: `prev` moved to node(10), `pres` moved to node(20). (Key 20 < 30) is true.



Iteration 2: `prev points to node(20), pres moved to node(50). Loop condition check: (Key 50 < 30) is false. Looo condition fails.



Loop Exits.
Final Pointers `prev` and `pres` confirm position (between 20 and 50).
Inserted node(30) at position.

```
//  mylist.h

#ifndef MYLIST_H

#defne MYLIST_H

struct node

{

	int key_;

	struct node *next_;

};

typedef struct node node_t;

struct mylist

{

	node_t *head;

};

typedef struct mylist mylist_t;

void init_list(mylist_t*);

void insert_list(mylist_t*, int key_);

void disp_list(mylist_t*);

void free_list(mylist_t*);

#endif
```

```
// mylist.c

#include <stdio.h>

#include "mylist.h"

#include <stdlib.h>

void init_list(mylist_t* ptr_list)

{

	ptr_list->head = NULL;

}

// 1. empty list

// 2. insert in the middle

// 3. insert in the beginning

// 4. insert at the end

void insert_list(mylist_t* ptr_list, int key)

{

	// create the node

	node_t* temp = (node_t*)malloc(sizeof(node_t));
```

```
temp->key_ = key; temp->next_ = NULL;

// 1.empty list

if(ptr_list->head == NULL)

{

        ptr_list->head = temp;

        temp->next_ = NULL;

}

else

{

        // traverse

        node_t* prev = NULL;

        node_t* pres = ptr_list->head;

        while(pres != NULL && pres->key_ < temp->key_ )

                {

                prev = pres;

                pres = pres->next_;

                }

        if(prev == NULL) // insert in the beginning

        {

                temp->next_ = pres;

                ptr_list->head = temp;

        }
```

```
                else // middle or end
                {
                        temp->next_ = pres;
                        prev->next_ = temp;
                }
        }
}
```

```
void disp_(node_t* q)
{
	while(q != NULL)
	{
		printf("%d ", q->key_);
		q = q->next_;
	}
}
void disp_list(mylist_t* ptr_list)
{
	disp_(ptr_list->head);
}
void free_list_(node_t* q)
{
	node_t* r;
	while(q != NULL)
	{
		r = q->next_;
		free(q);
		q = r;
	}
}
```

```
void free_list(mylist_t* ptr_list)
{
	free_list_(ptr_list->head);
}
```

We have been successful in separating the interface from the implementation. The client can use it independently without worrying about the changes to the implementation.

How do we capture the dependency between files and the commands to be executed when the dependent files change? The tool used for this purpose is called a build tool. **Make** build automation tool was designed and created by **Stuart Feldman** at Bell Labs in **April 1976**. In mingw, there is a similar tool called mingw32-make. In Ubuntu and Mac, the tool is called make.

Create a file – called makefile by default.

Indicate the binary – output – file then : followed by names of all files on which this target depends.

In the next line, start with a tab and indicate the commands to be executed to make the target.

Repeat this for every target file.

Makefile :

```
a.exe : mylist.o client.o
	gcc mylist.o client.o
mylist.o : mylist.c mylist.h
```

```
	gcc -Wall -c mylist.c

client.o : client.c mylist.h

	gcc -Wall -c client.c
```

To execute the makefile, say

Mingw32-make

The makefile holds dependency information and executes only those commands to remake the target when the files on which the target depends change.

So surprisingly, this tool has not changed for over 55 years!